

Test Technique : Data Engineer

Nous vous remercions pour votre intérêt pour le poste de Data Engineer au sein de notre équipe.

Ce test technique a pour objectif d'évaluer vos compétences en ingénierie de données : génération de données réalistes, transformation, pipeline incrémental et containerisation.

Contexte

Pour utiliser les services Chargemap Business, les clients professionnels doivent posséder des licences pour leurs badges de recharge ou les bornes installées sur leurs parkings. Ces licences sont renouvelées chaque année.

Nous souhaitons, sur la base de données simulées, construire un pipeline de données robuste permettant d'analyser l'évolution des licences dans le temps.

Partie 1 : Génération de Données Réalistes (Python)

Objectif : Générer des données qui simulent un comportement métier réaliste.

Vous devez créer deux fichiers CSV :

1.1 Fichier `initial_licenses.csv`

Colonne	Description
<code>id</code>	Identifiant unique (1 à 100)
<code>customer_id</code>	Identifiant client (plusieurs licences peuvent appartenir au même client)
<code>type</code>	Type de licence : <code>PASS</code> , <code>SIM</code> , ou <code>SUPERVISION</code>
<code>creation_date</code>	Date de création (entre le 1er janvier 2023 et aujourd'hui)
<code>price</code>	Prix annuel (entre 0 et 5000)
<code>renewable</code>	Booléen indiquant si la licence est active/renouvelable

1.2 Fichier `license_changes.csv`

Colonne	Description
<code>id</code>	Identifiant unique de l'événement

Colonne	Description
<code>license_id</code>	Référence à <code>id</code> dans <code>initial_licenses.csv</code>
<code>date</code>	Date de la modification (après la date de création)
<code>price</code>	Nouveau prix
<code>type</code>	Nouveau type de licence
<code>renewable</code>	Nouvel état renouvelable

Minimum : 10 000 modifications.

Contraintes de réalisme

Vos données doivent refléter des comportements métier plausibles :

1. **Saisonnalité** : Les créations de licences sont plus fréquentes en début d'année (Q1) — période de renouvellement des contrats.
2. **Distribution des clients** : La plupart des clients possèdent 1 à 2 licences, mais quelques grands comptes en possèdent beaucoup plus (distribution réaliste type Pareto).
3. **Cohérence des modifications** :
 - Les augmentations de prix sont plus fréquentes que les baisses.
 - Les changements de type suivent une logique (ex : upgrade de PASS vers SUPERVISION plus fréquent que l'inverse).
 - Certaines licences ont de nombreuses modifications, la majorité en ont peu.
4. **Cohérence temporelle** : Aucune modification ne peut précéder la date de création de la licence.

Livrable attendu

- Script Python commenté
- Expliquez brièvement (en commentaire ou README) les choix de distribution et de logique métier

Partie 2 : Transformation et Validation (Python & SQL)

2.1 Validation des données

Avant toute transformation, écrivez un script de validation qui vérifie :

- Aucune modification avant la date de création d'une licence
- Pas d'identifiants dupliqués
- Tous les `license_id` existent dans `initial_licenses.csv`
- Les types sont valides (`PASS` , `SIM` , `SUPERVISION`)
- Les prix sont dans la plage attendue
- Les distributions des caractéristiques sensibles

Livrable : Script qui journalise les anomalies détectées (même si vos données n'en contiennent pas).

2.2 Transformation des données

Générez un fichier CSV avec l'état agrégé des licences par jour et par type :

Colonne	Description
<code>date</code>	Date
<code>type</code>	Type de licence
<code>active_license_count</code>	Nombre de licences actives (renouvelables) à cette date
<code>active_license_price</code>	Prix total des licences actives à cette date
<code>inactive_license_count</code>	Nombre de licences inactives à cette date
<code>daily_active_diff</code>	Différence du nombre de licences actives vs veille
<code>daily_price_diff</code>	Différence du prix total actif vs veille
<code>daily_inactive_diff</code>	Différence du nombre de licences inactives vs veille

Règles :

- Une ligne par type de licence par jour, de la plus ancienne création jusqu'à aujourd'hui
- Les jours sans modification reprennent les valeurs de la veille (différences à 0)

Livrables attendus

1. **Script Python** réalisant cette transformation (en utilisant la lib pandas)

2. **Requête SQL** produisant le même résultat, en supposant les tables suivantes :

```
CREATE TABLE initial_licenses (  
    id INTEGER PRIMARY KEY,  
    customer_id INTEGER,  
    type VARCHAR(50),  
    creation_date DATE,  
    price INTEGER,  
    renewable BOOLEAN  
);  
  
CREATE TABLE license_changes (  
    id INTEGER PRIMARY KEY,  
    license_id INTEGER,  
    date DATE,  
    price INTEGER,  
    type VARCHAR(50),  
    renewable BOOLEAN  
);
```

3. Des éléments de réflexion portant sur l'architecture de la base de données qui gérera ces requêtes
-

Partie 3 : Containerisation et Tests

L'objectif est de rendre votre pipeline reproductible et fiable.

3.1 Containerisation (Docker)

Créez un environnement Docker permettant d'exécuter l'ensemble de votre pipeline.

Livrables attendus :

1. **Dockerfile** : Image contenant toutes les dépendances nécessaires pour exécuter vos scripts Python
2. **(Optionnel)** Une base de données pour tester vos requêtes SQL

Contraintes :

- L'image doit être légère (privilégiez une image de base appropriée)
- Les fichiers CSV générés doivent être accessibles depuis l'hôte (volumes)
- Le pipeline complet doit pouvoir être lancé dans docker

3.2 Tests Automatisés (pytest)

Écrivez des tests unitaires et d'intégration pour valider votre code.

Tests attendus :

Type	Exemples
Tests unitaires	Fonctions de validation, calculs d'agrégation, logique métier
Tests d'intégration	Pipeline complet sur un jeu de données réduit
Tests de cohérence	Les données générées respectent les contraintes de réalisme

Contraintes :

- Utilisez `pytest` comme framework de test
- Couvrez au minimum les cas nominaux et quelques cas limites
- Les tests doivent pouvoir s'exécuter dans le container Docker

Livrables attendus

- Un dossier `tests/` avec vos fichiers de tests et instructions pour les exécuter.

3.3 Question ouverte

Comment intégreriez-vous ces tests dans un workflow de développement ?
 Décrivez brièvement ce que vous mettriez en place si vous aviez accès à GitHub Actions ou GitLab CI (pas besoin de l'implémenter, juste décrire les étapes du pipeline CI).

Partie 4 : Pipeline Incrémental

Votre pipeline de transformation doit maintenant s'exécuter toutes les heures en production. Retraiter l'ensemble des données à chaque exécution n'est pas viable (trop lent, trop coûteux).

4.1 Conception

Décrivez votre approche pour un pipeline incrémental :

- Comment suivez-vous ce qui a déjà été traité ? (watermark, checkpoint, curseur...)
- Comment mettez-vous à jour les agrégations existantes sans tout recalculer ?
- Quel état persistez-vous entre les exécutions ?

4.2 Implémentation

Implémentez une version simplifiée en Python qui :

- Lit uniquement les nouvelles modifications depuis le dernier run
- Met à jour le fichier d'agrégation quotidienne
- Peut être relancée sans créer de doublons (idempotence)

4.3 Gestion des cas limites

Expliquez comment vous géreriez les situations suivantes. Si possible, intégrez ces cas dans votre implémentation :

Cas limite	Description
Données tardives	Une modification datée du 15 janvier arrive le 20 janvier. Comment recalculer les agrégations impactées ?
Reprise sur erreur	Le pipeline échoue à mi-parcours. Comment garantir une reprise propre sans corruption de données ?
Backfill	On vous demande de recalculer les 30 derniers jours suite à une correction de bug. Comment procédez-vous ?

4.4 Question ouverte

Quels outils ou frameworks utiliseriez-vous en production pour ce type de pipeline ? (ex : Airflow, dbt, Spark, Kafka, Dagster...)

Justifiez votre choix en fonction du contexte (volume de données, fréquence, équipe, infrastructure existante).

Soumission

Envoyez une archive contenant :

- ☐ Scripts Python (génération, validation, transformation, pipeline incrémental)

- ☐ Fichiers CSV générés
- ☐ Requêtes SQL (fichier `.sql`)
- ☐ `Dockerfile`
- ☐ Dossier `tests/` avec vos tests pytest
- ☐ `requirements.txt`
- ☐ `README.md` expliquant votre démarche, vos choix et répondant aux questions posées
- ☐ Historique Git recommandé (commits réguliers appréciés)

N'hésitez pas à nous contacter si vous avez des questions concernant ce test.